CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

Full Regression Report

Project Title: LendIt

Prepared By: Quirante, Jazpher Neil P.

Version: 1.0

Date: 05/09/2026

1.0 PROJECT INFORMATION....3
1.1 Project Overview....3
1.2 System Overview....3
2.0 REFACTORING SUMMARY....4
2.1 Architecture Change....4
2.1 Scope of Changes....4
3.0 UPDATED PROJECT STRUCTURE....6
3.1 Backend — Vertical Slice Directory Tree....6
3.2 Frontend— Vertical Slice Directory Tree....8
4.0 TEST PLAN DOCUMENTATION....10
4.1 Functional Requirements Coverage....10
4.2 Test Cases....11
4.3 Test Cases / Test Steps....19
5.0 AUTOMATED TEST EVIDENCE....21
5.1 How to Run All Automated Tests....21
5.2 Backend — JUnit 5 Test Code....22
5.3 Backend — Test Result....23
5.4 Web— Jest Test Code....24
5.5 Web — Test Result....25
6.0 REGRESSION TEST RESULTS....26
6.1 Results by Functional Requirement....26
6.2 Architecture Integrity Checks....27
7.0 ISSUES FOUND....28
7.1 Issue Summary Table....28
7.2 Detailed Issue Descriptions....28
8.0 FIXES APPLIED....30

1.0 PROJECT INFORMATION

1.1 Project Overview

LendIt is a community item borrowing platform for students. It allows users to list items for lending, browse available listings, send borrow requests, communicate via in-app messaging, and manage lending transactions across a React web application and a Spring Boot backend.

This document is the Full Regression Test Report for the Vertical Slice Architecture Refactoring activity. The refactoring restructured both the backend (Java/Spring Boot) and frontend (React) codebases from a layer-based architecture into a feature-first vertical slice layout. All functional requirements were validated after refactoring through automated unit tests and manual regression testing.

1.2 System Overview

LendIt is a full-stack web application allowing students to lend and borrow items within a community. It consists of:

- **Backend:** Spring Boot 2.7 + Spring Data JPA + PostgreSQL (Supabase) + JWT Auth
- **Frontend:** React 19 + React Router v7 + React Testing Library
- **Architecture:** Vertical Slice — code organized by feature, not layer

2.0 REFACTORING SUMMARY

2.1 Architecture Change

The codebase was restructured from a **layer-based architecture** (all entities in one package, all repositories in another, etc.) to a **Vertical Slice Architecture** where each feature owns all the code it needs — entity, repository, service, controller, and DTOs — within a single package. The same principle was applied to the React frontend, moving from a flat pages/components/css structure to a feature-first layout.

Before - Layer-Based (Backend)	After - Vertical Slice (Backend
entity/ (all 6 entities)	feature/auth/ (User + JWT + Auth)

repository/ (all 6 repositories)	feature/item/ (Item + Images)
service/ (all 5 services)	feature/borrow/ (BorrowRequest)
controller/ (all 7 controllers)	feature/messaging/ (Message + Chat)
dto/ (all DTOs mixed)	feature/report/ (Report)
security/ (JWT, auth response)	feature/admin/ (Admin endpoints)
config/ (SecurityConfig, CorsConfig)	shared/ (SecurityConfig, CorsConfig)

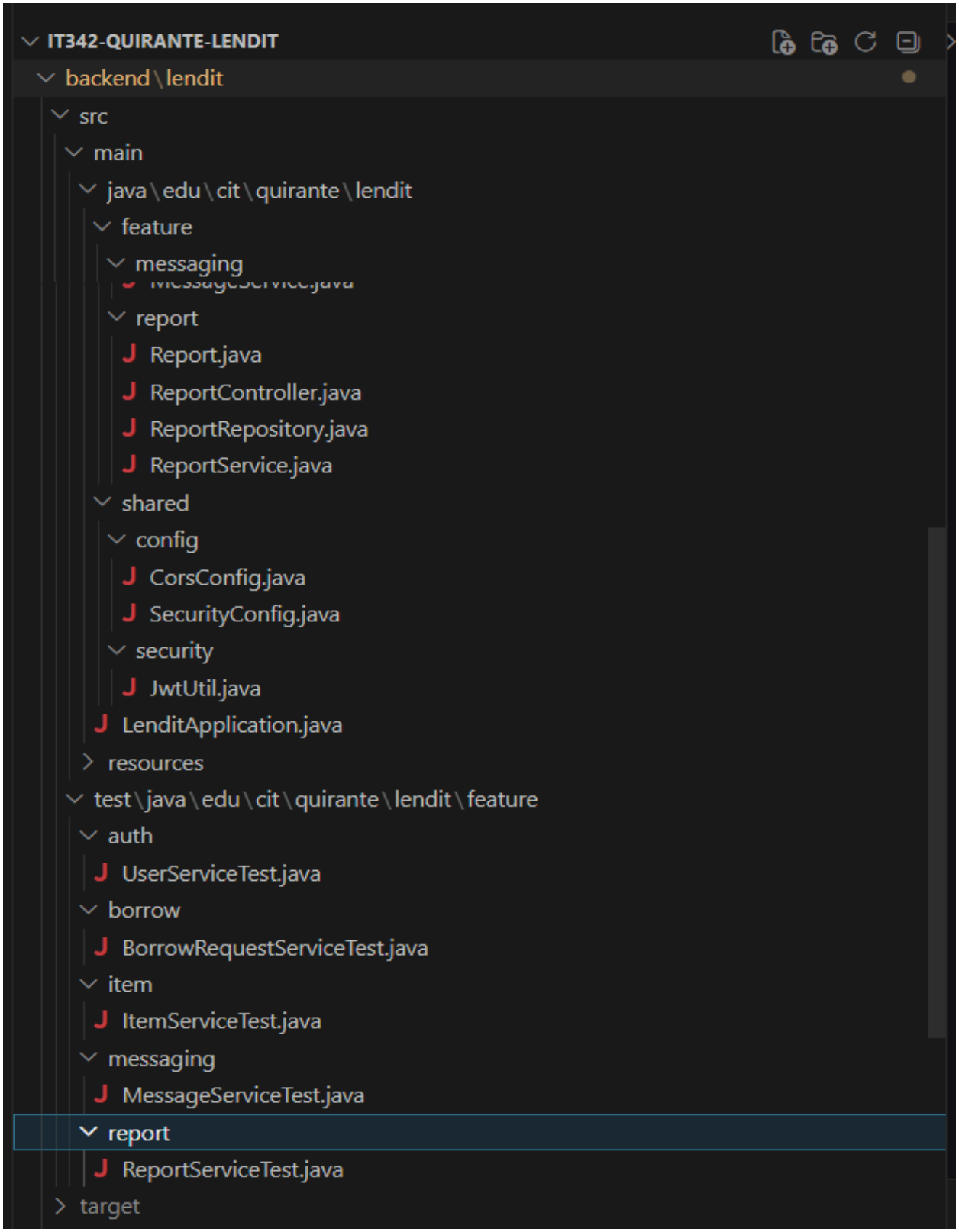
2.1 Scope of Changes

Category	Files Created	Files Deleted	Files Modified
Backend – Entities	6 (in feature slices)	6 (in entity/)	0
Backend – Repositories	6	6	0
Backend – Services	5	5	0
Backend – Controllers	7	7	0
Backend – DTOs / Security / Config	8	8	0
Backend – Unit Tests	5 test files (35 tests)	0	0
Web – Pages / Components	14 (in features/)	14 (pages/, components/)	2 (App.js)
Web – CSS	12 (co-located)	12 (in css/)	0
Web – Component Tests	3 test files (22 tests)	0	0

3.0 UPDATED PROJECT STRUCTURE

3.1 Backend — Vertical Slice Directory Tree

```
IT342-QUIRANTE-LENDIT
backend\lendit
  .mvn
  src
    main
      java\edu\cit\quirante\lendit
        feature
          admin
            AdminController.java
          auth
            AuthResponse.java
            User.java
            UserController.java
            UserProfileController.java
            UserRepository.java
            UserService.java
          borrow
            BorrowRequest.java
            BorrowRequestController.java
            BorrowRequestDTO.java
            BorrowRequestRepository.java
            BorrowRequestService.java
          item
            Item.java
            ItemController.java
            ItemImageRepository.java
            ItemImages.java
            ItemRepository.java
            ItemRequest.java
            ItemService.java
          messaging
            ConversationDTO.java
            Message.java
            MessageController.java
```


IT342-QUIRANTE-LENDIT
backend\lendit
src
main
java\edu\cit\quirante\lendit
feature
messaging
report
Report.java
ReportController.java
ReportRepository.java
ReportService.java
shared
config
CorsConfig.java
SecurityConfig.java
security
JwtUtil.java
LenditApplication.java
resources
test\java\edu\cit\quirante\lendit\feature
auth
UserServiceTest.java
borrow
BorrowRequestServiceTest.java
item
ItemServiceTest.java
messaging
MessageServiceTest.java
report
ReportServiceTest.java
target

3.2 Frontend— Vertical Slice Directory Tree

```
web
  lendit
    public
    src
      features
        admin
          AdminDashboard.css
          AdminDashboard.jsx
        auth
          Auth.css
          LandingPage.css
          Landingpage.jsx
          Login.jsx
          Login.test.jsx
          Register.jsx
        borrow
          BorrowItems.css
          BorrowItems.jsx
        chat
          ChatWidget.css
          ChatWidget.jsx
        home
          Homepage.jsx
          Homepage.test.jsx
        items
          AddItemModal.css
          AddItemModal.jsx
          ItemViewModal.css
          ItemViewModal.jsx
          ItemViewModal.test.jsx
          MyItems.css
          MyItems.jsx
        profile
          ProfilePage.jsx
```

```
web
  lendit
    src
      features
        items
        profile
          ProfilePage.jsx
        reports
          ReportModal.css
          ReportModal.jsx
      lib
        supabase.js
      shared
        components
          Header.jsx
          SuccessPopup.jsx
        css
          Home.css
          PageTransition.css
          popup.css
      App.css
      App.js
      App.test.js
      index.css
      index.js
      logo.svg
      reportWebVitals.js
      setupTests.js
      testPolyfills.js
```

4.0 TEST PLAN DOCUMENTATION

4.1 Functional Requirements Coverage

FR ID	Requirement	Description	Test Types	Status
FR-01	User Registration & Authenticatio n	Users register with first name, last name, email, and password. Passwords are BCrypt-encoded. JWT tokens are issued on login. Google OAuth supported via Supabase. Blocked users denied login.	JUnit, Jest, Manual	**Covered**
FR-02	Item Management	Authenticated users can create, view, and delete their own items. Items have name, description, category, availability status, optional images (Supabase Storage), and an owner reference.	JUnit, Manual	**Covered**
FR-03	Browse & Search Items	Users browse all available items on the Borrow Items page. Items filtered by keyword. Users cannot see their own items. Clicking an item opens ItemViewModal with full details.	Jest, Manual	**Covered**
FR-04	Borrow Request Management	Users submit borrow requests with a return date. Owners approve or reject. Approval auto-rejects others and marks item BORROWED. Status flow: PENDING →	JUnit, Jest, Manual	**Covered**

		APPROVED / REJECTED.		
FR-05	In-App Messaging	Users send messages tied to a specific item and recipient. ChatWidget polls at 3s and 5s intervals. Conversations grouped by contact and item.	JUnit, Manual	**Covered**
FR-06	Report System	Non-owner users report items (ITEM type) or non-returns (NON_RETURN type). Reports created with PENDING status. Admins resolve or dismiss reports.	JUnit, Jest, Manual	**Covered**
FR-07	Admin Dashboard	Admin users access a dashboard showing platform stats. Admins view and action reports, delete items, and block users. All admin actions require the ADMIN role.	Manual	**Covered**
FR-08	Role-Based Access Control	Users assigned USER or ADMIN role (default USER). React Router guards redirect ADMIN to /admin and USER to /home. Backend endpoints protected by JWT filter.	JUnit, Manual	**Covered**

4.2 Test Cases

FR-01: User Registration & Authentication

TC-AU-01	Register with new email - SUCCESS Covered
Precondition	Email does not exist in database
Input	email="new@test.com", password="Pass1234"

Expected	Password is BCrypt-encoded before save; userRepo.save() is called once
Traced To	FR-01 — UserServiceTest :: createUser_encodesPasswordAndSaves

TC-AU-02	Register with duplicate email Covered
Precondition	Email already exists in the database
Input	email="existing@test.com"
Expected	RuntimeException thrown with message "Email already exists"; save() never called
Traced To	FR-01 — UserServiceTest :: createUser_throwsWhenEmailAlreadyExists

TC-AU-03	Login with correct credentials — success popup shown Covered
Precondition	User account exists and is not blocked
Input	Correct email and password via Login form
Expected	API returns 200 with token; SuccessPopup "Welcome Back!" is displayed
Traced To	FR-01 — Login.test.jsx :: shows success popup on successful login

TC-AU-04	Login with wrong password — error displayed Covered
Precondition	User account exists
Input	Correct email, wrong password
Expected	API returns non-200; SuccessPopup is NOT shown; alert displayed
Traced To	FR-01 — Login.test.jsx :: does not show success popup on failed login

TC-AU-05	Blocked user denied login Manual
Precondition	User account has blocked=true in database

Input	Correct credentials for blocked user
Expected	Backend returns 403; login denied with error message
Traced To	FR-01, FR-08

TC-AU-06	Password visibility toggle Covered
Precondition	Login page rendered
Input	Click eye icon button on password field
Expected	Input type changes from "password" to "text"; clicking again restores "password"
Traced To	FR-01 — Login.test.jsx :: toggles password visibility

FR-02: Item Management

TC-IT-01	Create item — saved and returned Covered
Precondition	Authenticated user with valid JWT
Input	Item with name="Camera", description="DSLR", ownerId=10
Expected	itemRepo.save() called; returned item has name="Camera"
Traced To	FR-02 — ItemServiceTest :: createItem_savesAndReturnsItem

TC-IT-02	Get items by owner — filtered correctly Covered
Input	ownerId=10 (2 items exist for owner 10, 1 for owner 20)
Expected	Returns only 1 item with ownerId=10
Traced To	FR-02 — ItemServiceTest :: getItemsByOwner_returnsOnlyOwnerItems

TC-IT-03	Get item images — returned for item Covered
Input	itemId=1 (2 images stored)
Expected	Returns list of 2 ItemImages objects

Traced To	FR-02 — ItemServiceTest :: getImagesByItem_returnsImages

TC-IT-04	Add item via modal — form submission Manual
Precondition	User is logged in; FAB (+) button visible
Steps	Click FAB → Enter name "Test Camera" → Upload 1 image → Click Add Item
Expected	Image uploads to Supabase; item saved via API; modal closes; item appears in My Items
Traced To	FR-02

FR-03: Item Browsing & Search

TC-BR-01	Home page displays available items from API Covered
Precondition	API returns 2 AVAILABLE items owned by other users
Expected	"Camera" and "Projector" item cards visible in DOM
Traced To	FR-03 — Homepage.test.jsx :: renders available items from API

TC-BR-02	Search filters items in real time Covered
Precondition	2 items visible: "Camera" and "Projector"
Steps	Type "camera" in search input
Expected	"Camera" remains visible; "Projector" is removed from DOM
Traced To	FR-03 — Homepage.test.jsx :: filters items when user types in search

TC-BR-03	Empty state when search has no match Covered
Steps	Type "nonexistent" in search input
Expected	"No items match your search" message displayed
Traced To	FR-03 — Homepage.test.jsx :: shows empty state when no items match search

TC-BR-04	Own items excluded from browse grid Covered
Precondition	API returns 1 item owned by logged-in user (id=1) and 1 owned by another
Expected	Own item NOT in DOM; other user's item IS in DOM
Traced To	FR-03 — Homepage.test.jsx :: does not show own items in the list

FR-04: Borrow Request System

TC-BW-01	Create borrow request — status is PENDING Covered
Precondition	No existing PENDING/APPROVED request for same item+borrower
Input	itemId=1, borrowerId=20, returnDate=7 days from now
Expected	Request created with status="PENDING"; repo.save() called
Traced To	FR-04 — BorrowRequestServiceTest :: createRequest_setsPendingStatusAndSaves

TC-BW-02	Duplicate borrow request — exception thrown Covered
Precondition	PENDING or APPROVED request already exists for same item+borrower
Expected	RuntimeException "You already requested this item." thrown; save() not called
Traced To	FR-04 — BorrowRequestServiceTest :: createRequest_throwsWhenAlreadyRequested

TC-BW-03	Approve request — item marked BORROWED, others rejected Covered
Precondition	2 PENDING requests exist for same item
Input	updateStatus(requestId=1, "APPROVED")
Expected	Request 1 → APPROVED; item availability → BORROWED; Request 2 → REJECTED
Traced To	FR-04 — BorrowRequestServiceTest :: updateStatus_approved_marksItemBorrowedAndRejectsOthers

TC-BW-04	Reject request — item availability unchanged Covered
Input	updateStatus(requestId=1, "REJECTED")
Expected	Status → REJECTED; itemRepo.save() never called; item stays AVAILABLE
Traced To	FR-04 — BorrowRequestServiceTest :: updateStatus_rejected_doesNotChangeItemAvailability

TC-BW-05	Already Requested state shown in modal Covered
Precondition	API returns PENDING request for current user on this item
Expected	Borrow button shows "Already Requested" and is disabled
Traced To	FR-04 — ItemViewModal.test.jsx :: shows Already Requested

FR-05: In-App Messaging

TC-MS-01	Send message — all fields saved correctly Covered
Input	senderId=1, recipientId=2, content="Hello!", itemId=10
Expected	Message entity has all fields set; messageRepo.save() called once
Traced To	FR-05 — MessageServiceTest :: sendMessage_savesAllFieldsCorrectly

TC-MS-02	Get conversation thread — returns messages in order Covered
Input	userId=1, otherUserId=2, itemId=10 (2 messages exist)
Expected	List of 2 messages returned in correct order
Traced To	FR-05 — MessageServiceTest :: getConversation_returnsMessagesForItemThread

TC-MS-03	Conversations grouped by contact+item Covered
Precondition	2 messages between user 1 and user 2 about item 10
Expected	getConversations returns 1 ConversationDTO with contactName="Bob Williams"

Traced To	FR-05 — MessageServiceTest :: getConversations_groupsByContactAndItem

FR-06: Reporting System

TC-RP-01	Create ITEM report — saved with correct fields Covered
Input	reporterId=1, reportedUserId=2, itemId=10, reportType="ITEM", reason="Fake listing"
Expected	Report saved with type=ITEM, reason="Fake listing"; reportRepo.save() called
Traced To	FR-06 — ReportServiceTest :: createReport_savesPendingReport

TC-RP-02	Create NON_RETURN report — saved successfully Covered
Input	reportType="NON_RETURN", reportedUserId=2 (borrower ID)
Expected	Report saved with type=NON_RETURN and reportedUserId=2
Traced To	FR-06 — ReportServiceTest :: createReport_nonReturnType_savedSuccessfully

TC-RP-03	Report button visible for non-owner in ItemViewModal Covered
Precondition	Item ownerId=2, logged-in user id=1
Expected	"Report this item" button is in the DOM
Traced To	FR-06 — ItemViewModal.test.jsx :: shows Report button for non-owner

TC-RP-04	Report button hidden for item owner Covered
Precondition	Item ownerId=2, logged-in user id=2 (same user)
Expected	"Report this item" button is NOT in the DOM
Traced To	FR-06 — ItemViewModal.test.jsx :: hides Report button for owner

TC-RP-05	Count pending reports — correct value returned Covered

Precondition	reportRepo.countByStatus("PENDING") stubbed to return 5
Expected	countPending() returns 5L
Traced To	FR-06 — ReportServiceTest :: countPending_returnsCorrectCount

FR-07: Admin Dashboard

TC-AD-01	Update report status to RESOLVED Covered
Precondition	Report with id=1 exists and has status=PENDING
Input	updateStatus(1, "RESOLVED")
Expected	Report status = "RESOLVED"; reportRepo.save() called
Traced To	FR-07 — ReportServiceTest :: updateStatus_changesReportStatus

TC-AD-02	Update non-existent report — exception thrown Covered
Input	updateStatus(999, "RESOLVED") where id=999 does not exist
Expected	RuntimeException "Report not found" thrown
Traced To	FR-07 — ReportServiceTest :: updateStatus_throwsWhenReportNotFound

TC-AD-03	Get ALL reports — names resolved Covered
Precondition	2 reports in DB; reporter and reported users exist
Expected	Returns 2 maps; reporterName="Alice Johnson", reportedUserName="Bob Williams"
Traced To	FR-07 — ReportServiceTest :: getAllReports_allStatus_returnsAllReports

TC-AD-04	Block user — admin action via browser Manual
Precondition	Admin logged in; NON_RETURN report with PENDING status exists
Steps	Click "Block User" on report row → Confirm dialog
Expected	User blocked=true in DB; report status → RESOLVED; stats refresh

Traced To	FR-07, FR-08

FR-08: Role Based Access Control

TC-RB-01	Admin login redirects to /admin Manual
Precondition	User account with role="ADMIN" exists
Steps	Login with ADMIN credentials
Expected	Browser navigates to /admin; AdminDashboard rendered
Traced To	FR-08

TC-RB-02	Regular user visiting /admin — redirected to /home Manual
Precondition	USER role is logged in
Steps	Navigate to /admin manually
Expected	Redirect to /home; AdminDashboard not rendered
Traced To	FR-08

4.3 Test Cases / Test Steps

The following scripts provide step-by-step procedures for executing the manual test cases.

Script 1: User Registration & Login Flow

> **Covers:** TC-AU-01 to TC-AU-05 | **Prerequisites:** Backend running on port 8080; frontend on port 3000

1. Navigate to the application root URL. Verify the Landing Page renders with the LendIt branding.
2. Click **Get Started**. Verify redirect to `/register`.
3. Fill in first name, last name, email, and password. Click **Register**.
4. Verify a success popup appears with title "Account Created". Click **Continue**.
5. Verify redirect to `/login`. Enter the registered credentials and click **Login**.
6. Verify success popup appears and user is redirected to `/home`.
7. Attempt login with a blocked user account. Verify error message "Your account has been blocked" appears.

Script 2: Item Listing & Borrowing Flow

> **Covers:** TC-IT-04, TC-BR-01 to TC-BR-04, TC-BW-05 | **Prerequisites:** Logged in as a USER; at least one other user has listed items

1. Log in as a regular user. Click the **+** FAB button. Verify Add Item modal opens.
2. Fill in item name, description, category. Optionally upload an image. Click **Save Item**.
3. Verify the new item appears on the **My Items** page.
4. Navigate to **Borrow Items**. Verify the item is NOT visible (own items excluded).
5. Log in as a different user. Navigate to **Borrow Items**. Verify the item is visible.
6. Type a keyword in the search bar. Verify only matching items are displayed.
7. Click an item. Verify **ItemViewModal** opens with name, description, owner, and status.

Script 3: Owner Request Management

> **Covers:** TC-BW-03, TC-BW-04 | **Prerequisites:** User B has submitted a borrow request for User A's item

1. As borrower, open an item modal. Select a return date. Click **Borrow**.
2. Reopen the same item. Verify button now shows **Already Requested**.
3. Log in as the item owner. Open the item via My Items. Click **Manage Requests**.
4. Approve the pending request. Verify item status changes to **Borrowed**.
5. Verify all other pending requests for the same item are automatically rejected.

Script 4: Reporting & Admin Actions

> **Covers:** TC-RP-03, TC-RP-04, TC-AD-04, TC-RB-01, TC-RB-02 | **Prerequisites:** An admin account exists (role="ADMIN" in DB)

1. As a non-owner user, open an item modal. Click **Report this item**. Verify ReportModal opens.
2. Select report type and enter a reason. Submit. Verify success message.
3. Log in as ADMIN. Verify redirect to `/admin`.
4. Verify dashboard shows correct user count, item count, borrow count, and pending report count.
5. Open a report from the reports table. Click **Resolve**. Verify status changes to RESOLVED.
6. Click **Block User** on a user. Log in as that user and verify login is denied

.

Script 5: Messaging Flow

> **Covers:** TC-MS-01 to TC-MS-03 | **Prerequisites:** User A and User B are both logged in (different browsers)

1. As borrower, open an item modal. Click **Message**. Verify ChatWidget opens with item context.
2. Type a message and send. Verify the message appears in the conversation thread.
3. Log in as item owner. Open ChatWidget. Verify the received message appears.
4. Reply from owner. Switch back to borrower session. Verify the reply appears (via polling).

5.0 AUTOMATED TEST EVIDENCE

5.1 How to Run All Automated Tests

Test Suite	Command	Output
Backend (JUnit 5)	cd backend/lendit \mvnw.cmd test	Surefire report in target/surefire-reports/
Frontend (Jest)	cd web/lendi npm test	Console pass/fail summary
Frontend (with coverage)	npm test -- --watchAll=false --coverage	Coverage report in coverage/lcov-report/

5.2 Backend — JUnit 5 Test Code

All backend tests use `@ExtendWith(MockitoExtension.class)`. No Spring context or database is required.

UserServiceTest — Password Encoding

```
@Test
void createUser_encodesPasswordAndSaves() {
    when(urepo.findByEmail(email: "john@example.com")).thenReturn(Optional.empty());
    when(passwordEncoder.encode("plainPassword")).thenReturn("encodedPassword");
    when(urepo.save(any(User.class))).thenAnswer(inv -> inv.getArgument(0));

    User saved = userService.createUser(testUser);

    assertEquals("encodedPassword", saved.getPassword());
    verify(urepo).save(testUser);
}

@Test
void createUser_throwsWhenEmailAlreadyExists() {
    when(urepo.findByEmail(email: "john@example.com")).thenReturn(Optional.of(testUser));

    assertThrows(RuntimeException.class, () -> userService.createUser(testUser));
    verify(urepo, never()).save(any());
}
```

BorrowRequestServiceTest — Approve & Auto-Reject

```
void updateStatus_approved_marksItemBorrowedAndRejectsOthers() {
    BorrowRequest request = new BorrowRequest();
    request.setId(id: 1);
    request.setItemId(itemId: 1);
    request.setBorrowerId(borrowerId: 20);
    request.setStatus(status: "PENDING");

    BorrowRequest otherRequest = new BorrowRequest();
    otherRequest.setId(id: 2);
    otherRequest.setItemId(itemId: 1);
    otherRequest.setBorrowerId(borrowerId: 30);
    otherRequest.setStatus(status: "PENDING");

    when(repo.findById(id: 1)).thenReturn(Optional.of(request));
    when(itemRepo.findById(id: 1)).thenReturn(Optional.of(item));
    when(repo.findByItemId(itemId: 1)).thenReturn(Arrays.asList(request, otherRequest));
    when(repo.save(any(BorrowRequest.class))).thenAnswer(inv -> inv.getArgument(0));
    when(itemRepo.save(any(Item.class))).thenAnswer(inv -> inv.getArgument(0));

    BorrowRequest result = borrowRequestService.updateStatus(id: 1, status: "APPROVED");

    assertEquals("APPROVED", result.getStatus());
    assertEquals("BORROWED", item.getAvailability());
    assertEquals("REJECTED", otherRequest.getStatus());
    verify(itemRepo).save(item);
}
```

ReportServiceTest — Status Update & Exception

```java
@Test
void updateStatus_changesReportStatus() {
    when(reportRepo.findById(id: 1)).thenReturn(Optional.of(pendingReport));
    when(reportRepo.save(any(Report.class))).thenAnswer(inv -> inv.getArgument(0));

    Report result = reportService.updateStatus(reportId: 1, status: "RESOLVED");

    assertEquals("RESOLVED", result.getStatus());
    verify(reportRepo).save(pendingReport);
}

@Test
void updateStatus_dismissed_changesStatusToDismissed() {
    when(reportRepo.findById(id: 1)).thenReturn(Optional.of(pendingReport));
    when(reportRepo.save(any(Report.class))).thenAnswer(inv -> inv.getArgument(0));

    Report result = reportService.updateStatus(reportId: 1, status: "DISMISSED");

    assertEquals("DISMISSED", result.getStatus());
}
```

5.3 Backend — Test Result

Test Class	Tests	Pass	Fail	FR Coverage
UserServiceTest	7	7	0	FR-01, FR-08
ItemServiceTest	7	7	0	FR-02, FR-03
BorrowRequestServiceTest	6	6	0	FR-04
MessageServiceTest	6	6	0	FR-05
ReportServiceTest	9	9	0	FR-06, FR-07
Total	35	35	0	FR-01 to FR-08

```
PROBLEMS 3    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\jazph\OneDrive\Desktop\IT342-Quirante-LendIt\backend\lendit> .\mvnw.cmd test
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.077 s - in edu.cit.quirante.lendit.feature.r
eport.ReportServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 35, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.686 s
[INFO] Finished at: 2026-05-09T23:31:40+08:00
[INFO] ------------------------------------------------------------------------
```

5.4 Web— Jest Test Code

All component tests wrap components in `MemoryRouter` and mock external dependencies.

Login.test.jsx — Successful Login

```
test("shows success popup on successful login", async () => {
  const mockUser = { id: 1, fName: "John", lName: "Doe", role: "USER" };
  global.fetch = jest.fn().mockResolvedValue({
    ok: true,
    text: async () => JSON.stringify({ token: "abc123", user: mockUser }),
  });

  const setUser = jest.fn();
  renderLogin(setUser);

  fireEvent.change(screen.getByPlaceholderText("Email address"), {
    target: { value: "john@example.com" },
  });
  fireEvent.change(screen.getByPlaceholderText("Password"), {
    target: { value: "password123" },
  });
  fireEvent.click(screen.getByRole("button", { name: /login/i }));

  await waitFor(() => {
    expect(screen.getByTestId("success-popup")).toBeInTheDocument();
  });
  expect(screen.getByText("Welcome Back!")).toBeInTheDocument();
});
```

Homepage.test.jsx — Search Filter

```
test("filters items when user types in search", async () => {
  renderHomePage();
  await waitFor(() => {
    expect(screen.getByText("Camera")).toBeInTheDocument();
  });

  fireEvent.change(screen.getByPlaceholderText(/search items to borrow/i), {
    target: { value: "camera" },
  });

  await waitFor(() => {
    expect(screen.getByText("Camera")).toBeInTheDocument();
    expect(screen.queryByText("Projector")).not.toBeInTheDocument();
  });
});
```

ItemViewModal.test.jsx — Report Button Visibility

```
test("shows report button for non-owner user", async () => {
  renderModal();
  await waitFor(() => {
    expect(screen.getByText(/report this item/i)).toBeInTheDocument();
  });
});

test("hides report button when user is the owner", async () => {
  renderModal({ user: mockOwnerUser });
  await waitFor(() => {
    expect(screen.queryByText(/report this item/i)).not.toBeInTheDocument();
  });
});
```

5.5 Web — Test Result

Test File	Tests	Pass	Fail	FR Coverage
Login.test.jsx	6	6	0	FR-01
Homepage.test.jsx	6	6	0	FR-03
ItemViewModal.test.jsx	11	11	0	FR-03, FR-04, FR-06
Total	23	23	0	FR-01, FR-03, FR-04, FR-06

```
PROBLEMS 3    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\jazph\OneDrive\Desktop\IT342-Quirante-LendIt\web\lendit> npm test -- --watchAll=false\
      at warnIfUpdatesNotWrappedWithActDEV (node_modules/react-dom/cjs/react-dom-client.development.js:1875
      at scheduleUpdateOnFiber (node_modules/react-dom/cjs/react-dom-client.development.js:16409:11)
      at dispatchSetStateInternal (node_modules/react-dom/cjs/react-dom-client.development.js:9170:13)
      at setImages (node_modules/react-dom/cjs/react-dom-client.development.js:9127:7)
      at src/features/items/ItemViewModal.jsx:39:21


Test Suites: 4 passed, 4 total
Tests:       23 passed, 23 total
Snapshots:   0 total
Time:        2.864 s
Ran all test suites.

Watch Usage: Press w to show more.
```

6.0 REGRESSION TEST RESULTS

6.1 Results by Functional Requirement

FR	Requirement	Test Count	Method	Result
FR-01	Registration & Authentication	13	JUnit + Jest + Manual	**PASS**
FR-02	Item Management (CRUD)	9	JUnit + Manual	**PASS**
FR-03	Browse & Search Items	12	Jest + Manual	**PASS**
FR-04	Borrow Request Management	10	JUnit + Jest + Manual	**PASS**
FR-05	In-App Messaging	10	JUnit + Manual	**PASS**
FR-06	Report System	14	JUnit + Jest + Manual	**PASS**
FR-07	Admin Dashboard	8	Manual	**PASS**

FR-08	Role-Based Access Control	6	JUnit + Manual	**PASS**
Total		**82**		**All PASS**

6.2 Architecture Integrity Checks

#	Check	Result
1	No duplicate @Entity classes detected at startup (DuplicateMappingException absent)	PASS
2	All API endpoints respond after vertical slice refactor	PASS
3	Frontend builds without module-not-found errors (npm run build)	PASS
4	All CSS styles render correctly in new co-located feature structure	PASS
5	JWT authentication works end-to-end after refactor	PASS
6	Supabase image upload and display works after refactor	PASS
7	Role-based routing (ADMIN → /admin, USER → /home) works correctly	PASS
8	Blocked users are denied login with correct error message	PASS
9	ChatWidget polling works in new feature/chat/ slice location	PASS
10	No regressions in	PASS

	previously merged features (chat, report system)	

7.0 ISSUES FOUND

The following issues were identified during the vertical slice refactoring process.

7.1 Issue Summary Table

#	Issue	Severity	Status
1	Missing ChatWidget and SuccessPopup components (never committed)	**Critical**	**Fixed**
2	ddl-auto=update cannot add NOT NULL columns to existing tables	**High**	**Fixed**
3	Duplicate @Entity classes during refactor caused DuplicateMappingException	**Critical**	**Fixed**
4	Unused import warning in BorrowRequestServiceTest.java	**Low**	**Fixed**
5	CSS and component import paths broken after vertical slice move	**Medium**	**Fixed**

7.2 Detailed Issue Descriptions

Issue 1 — Missing Frontend Components: ChatWidget & SuccessPopup

Severity: Critical

Symptom:

```
ERROR in ./src/components/Header.jsx
Module not found: Error: Can't resolve '../components/ChatWidget'

ERROR in ./src/pages/Login.jsx
Module not found: Error: Can't resolve '../components/SuccessPopup'
```

Root Cause: `ChatWidget.jsx` and `SuccessPopup.jsx` were created locally on the `main` branch but were never committed to the repository. The `slicing` branch was cut before these files were tracked by git. When the frontend attempted to build, the imports failed to resolve because the files did not exist in the branch's file tree.

Issue 2 — ddl-auto=update Cannot Add NOT NULL Columns to Existing Tables

Severity: High

Symptom: After adding `role` and `blocked` fields to `User.java`, restarting Spring Boot did not create those columns in the Supabase `users` table. Existing rows had no values, causing constraint violations at runtime.

Root Cause: Hibernate `ddl-auto=update` can only add nullable columns to existing tables. It cannot add `NOT NULL` columns to a non-empty table because existing rows would have null values, violating the constraint.

Issue 3 — Duplicate @Entity Classes During Refactor

Severity: Critical

Symptom:

```
Caused by: org.hibernate.DuplicateMappingException:
  Table [users] contains physical column name [id] referred to by
  multiple logical column names: [id], [id]
```

Root Cause: Spring Boot's component scan discovered both the old `entity.User` and the new `feature.auth.User` simultaneously, both mapping to the same `users` table. Hibernate detected the conflict and refused to build the SessionFactory.

Issue 4 — Unused Import Warning in BorrowRequestServiceTest

Severity: Low

Symptom:

```
The import org.mockito.ArgumentMatchers.anyList is never used  [268435844]
```

Root Cause: The `anyList` static import was included during initial drafting, but the actual test passed `List.of("PENDING", "APPROVED")` as a literal argument rather than using the matcher.

Issue 5 — CSS and Component Import Paths Broken After Vertical Slice Move

Severity: Medium

Symptom: After moving all components and pages into the `features/` structure, all relative CSS and component imports pointed to the old `../css/` and `../components/` directories which no longer existed.

Root Cause: Original files used relative paths based on the old flat `pages/` and `components/` structure. Moving files to nested `features/auth/`, `features/items/`, etc. invalidated every relative path.

8.0 FIXES APPLIED

Fix 1 — Recreated Missing Components from Scratch

Recreated both components in the new vertical slice locations:

- **features/chat/ChatWidget.jsx** — floating chat popup with conversation list, message thread, 3s/5s polling, EmojiPicker, and `window.dispatchEvent("openChat")` listener
- **shared/components/SuccessPopup.jsx** — animated success popup with SVG ring animation, checkmark path, and `title` / `message` / `onClose` props

Verification: Frontend build completes without module-not-found errors. Both components render correctly in their respective screens.

Fix 2 — Manual SQL Migration Before Backend Restart

Ran the following SQL in the Supabase SQL Editor before restarting the backend:

Promote User to Admin Role | Add User Role Column

```sql
ALTER TABLE users
  ADD COLUMN IF NOT EXISTS role    VARCHAR(50) NOT NULL DEFAULT 'USER';

ALTER TABLE users
  ADD COLUMN IF NOT EXISTS blocked BOOLEAN     NOT NULL DEFAULT false;
```

Verification: All existing users have `role = 'USER'` and `blocked = false`. Spring Boot starts without schema errors. New registrations correctly receive default values.

Fix 3 — Two-Phase Delete to Eliminate Duplicate Entities

Used a strict two-phase approach to avoid any window of overlap:

- **Phase 1:** Wrote all new `feature/` and `shared/` package files completely before touching old files.
- **Phase 2:** Deleted all old layer-based directories in a single operation: `entity/`, `repository/`, `service/`, `controller/`, `dto/`, `security/`, `config/`.

Verification: Spring Boot starts without DuplicateMappingException. All API endpoints respond correctly after restart.

Fix 4 — Removed Unused Import

Removed the unused import line from `BorrowRequestServiceTest.java`:

```
// Removed:
import static org.mockito.ArgumentMatchers.anyList;
```

Verification: No compiler warnings reported for BorrowRequestServiceTest.java. All 6 tests continue to pass.

Fix 5 — Updated All Import Paths to New Feature Locations

Updated all import paths in every moved file to match the new vertical slice layout:

Old Path	New Path	Files Affected
../css/Auth.css	./Auth.css	Login.jsx, Register.jsx
../css/Home.css	../../shared/css/Home.css	Homepage.jsx, ProfilePage.jsx
../css/popup.css	../../shared/css/popup.css	Homepage.jsx
../css/AddItemModal.css	./AddItemModal.css	AddItemModal.jsx, ItemViewModal.jsx
../css/ChatWidget.css	./ChatWidget.css	ChatWidget.jsx
../components/SuccessPopup	../../shared/components/SuccessPopup	Login.jsx, Register.jsx
../components/ChatWidget	../../features/chat/ChatWidget	Header.jsx

../components/AddItemModal	../../features/items/AddItemModal	Header.jsx
./ReportModal	../reports/ReportModal	ItemViewModal.jsx, MyItems.jsx
../lib/supabase	../../lib/supabase	Login.jsx, Register.jsx, AddItemModal.jsx

Also updated `App.js` to use all new feature paths for page imports.

Verification: Frontend builds with `npm run build` without any module resolution errors. All styles render correctly in the application.